Company Hierarchy in a Software Firm

TechNova Pvt. Ltd. is a fast-growing software company. They have various roles, from junior developers to senior managers. To digitize their HR system, they need a simple object-oriented Java program that tracks:

- Basic employee info
- Role-specific responsibilities
- A clear hierarchy (Employee → Manager → SeniorManager)

The HR team wants this logic:

- **All employees** must show basic info — name and ID.
- **Managers** should mention which department they handle.
- **Senior Managers** should also mention how many team members they lead.

They also want to ensure that:

- No one should override the displayBasicInfo() function — it should remain consistent.
- The method overriding chain and use of super must be clear for auditability.

Class Breakdown

Base Class: Employee

- Fields: name, employeeId
- final method: displayBasicInfo() → Prints name & ID
- Method: getRole() → "Generic Employee"

Subclass: Manager

- Field: department
- getRole() → "Manager of [department] department"
- Method: displayDetails() →
  Calls super.getRole() → prints manager responsibilities

Subclass: SeniorManager (Multilevel)

- Field: teamSize
- Overrides getRole() → "Senior Manager managing [teamSize] team members in [department] department"
- displayDetails() →
  Calls super.displayDetails() → Adds "Senior strategic responsibilities"

- # Key Concepts Demonstrated

Feature	Where It's Shown
Multilevel Inheritance	Employee → Manager → SeniorManager
Method Overriding	getRole() in all 3 classes
super keyword	Used in constructors and method chaining
final method	displayBasicInfo() cannot be overridden
Polymorphism (if extended)	Could easily use Employee e = new SeniorManager(...)